



FORMULA 1 API

Javier Villalba y Álvaro Langa
Desarrollo de Aplicaciones Web

Índice

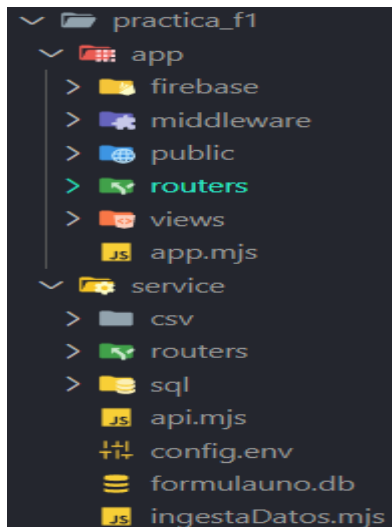
1. Concepto de la aplicación PAG 2
2. Estructura de la aplicación PAG 2
3. Flujo de la aplicación PAG 4
4. Ejecución de aplicación y explicación grafica PAG 11

1. Concepto de la aplicación

Nuestra aplicación web con interfaz permite a los usuarios consultar a información sobre toda la historia de la Formula 1 a través de una aplicación web que por detrás se comunica con una API. La base de datos que utilizamos para devolver los datos con la API es Turso y guardamos información de los pilotos, equipos, carreras y los distintos campeonatos.

La idea para poder monetizar esta aplicación sería que la información como la de todos los pilotos no se cobrara, pero si ya quieres algo mas específico como buscar a Fernando Alonso o algún otro piloto en especial que eso sea lo que se pudiera monetizar, es decir cosas más específicas como es un piloto en este caso se cobraría aparte.

2. Estructura de la aplicación



Principalmente tenemos la app (aplicación web) que tiene diferentes carpetas y su correspondiente archivo .mjs que es el que ejecutaremos.

- firebase: Exportamos la conexión y hacemos los métodos en los que comprobamos el login y registramos un usuario.
- middleware: Solo hay un middleware y es para la subida de fotos para el perfil.
- public: Esta carpeta la ponemos como estática dentro del app.mjs y las subcarpetas de estas son para temas como css y las imágenes.
- routes: Aquí tenemos todo lo que sería la “lógica” de una forma más organizada con las rutas de lectura, escritura, inserción, eliminación, etc...
Pero aquí sobre todo lo que hacemos son peticiones AXIOS a lo que es la API.
- views: Aquí tenemos las completas y las parciales, las completas serian algo más específicas y las parciales es algo que podemos ir incluyendo dentro de las completas cuando lo necesitemos.

Luego también tenemos la parte de la API que tiene también sus diferentes carpetas y archivos.

- csv: Tenemos los diferentes datos que luego utilizaremos para ingestar en la base de datos con el archivo ingestaDatos.mjs utilizando la librería de zebras.
- routers: Aquí es donde tenemos toda la lógica de verdad y aquí es donde la app web nos hace las peticiones.
- sql: Aquí hay diferentes archivos, un script para la creación de las tablas en la base de datos ya creada en turso, otro archivo con la estructura de las tablas como tal y luego la conexión que la exportamos para poder utilizarla en otros archivos.
- .env: Aquí hay valores ocultos que utilizamos para las conexiones por ejemplo de sql y firebase, para cuando lo ejecutamos con el `--env-file`.

3. Flujo de la aplicación

La explicación de estos puntos es en base a los pilotos pero la estructura seria la misma o muy parecida para los demás datos que gestionamos en nuestra aplicación web.

El método comprobarSesion() que veras en muchas partes del código es un código que comprueba si esta la sesión activa que queremos que es la de usuario.

```
export const comprobarSesion = (req) => {  
  const result = req.session.usuario  
  
  if(result){  
    return true  
  }  
  return false  
}
```

Para que un usuario se registre se hace todo desde la parte de aplicación web tanto la inserción del usuario en el firebase como en la base de datos SQL.

```
router.get("/register", (req, res) => {  
  if (!req.session.usuario) {  
    const htmlRegistro = path.join(path.resolve("../app/views/completes/"), "paginaRegistro.ejs")  
    res.render(htmlRegistro)  
  } else {  
    const htmlPrincipal = path.join(path.resolve("../app/views/completes/"), "paginaPrincipal.ejs")  
    res.render(htmlPrincipal)  
  }  
})  
  
router.post("/register", async (req, res) => {  
  const { email, password, nombreCompleto, userName, passwordConfirmar } = req.body  
  
  if (!email || !password || !nombreCompleto || !userName || passwordConfirmar !== password) {  
    console.log("Falta info")  
    return res.status(400).send("Falta información o las contraseñas no coinciden.")  
  } else {  
    try {  
      const saltRounds = 10  
      const hashPassword = await bcrypt.hash(password, saltRounds)  
  
      const result = await registrar(email, hashPassword)  
  
      if (!result) {  
        return res.status(400).send("El correo ya está registrado.")  
      }  
  
      const usuarios = `INSERT INTO usuarios (nombreCompleto, userName, email, passwordUsuario, rutaFoto) VALUES (?, ?, ?, ?, ?)`  
      await conn.execute(usuarios, [nombreCompleto, userName, email, hashPassword, "default.png"])  
  
      req.session.usuario = { email: email, tokens: 10000 }  
  
      res.redirect("/")  
    } catch (error) {  
      res.redirect("/register")  
    }  
  }  
})
```

Lo primero que vamos a explicar aquí es que flujo sigue al entrar o al intentar entrar con un usuario en nuestra aplicación web.

```
<form action="/login" method="post"> <!-- Se comprueba en la propia aplicación web -->
  <label for="email">Email</label>
  <input type="email" name="email" required>

  <label for="password">Contraseña</label>
  <input type="password" name="password" required>
  <button type="submit" name="iniciarSesion">Iniciar sesión</button>
</form>
<a href="/register">¿No tienes cuenta? Regístrate aquí</a>
```

Cuando entran a la pagina principal (localhost:3000) por el método GET (mediante la URL) se comprueba si tienen sesión lo primero y si tiene se le envía a la pagina principal y si no se le lleva al login como tal.

Cuando entra por post cogemos email y contraseña del formulario y se lo pasamos a este otro método que tenemos en conexionFirebase que es comprobarLogin y dependiendo de si existe ese usuario o no, hace una llamada AXIOS a la API con el parametro email introducido en el formulario para que nos devuelva los tokens guardados en Firebase e introducirlos en la sesión del usuario.

```
router.get("/login", async (req, res) => {
  if (!req.session.usuario) {
    const htmlLogin = path.join(path.resolve("../app/views/completes/"), "index.ejs")
    res.render(htmlLogin)
  } else {
    const htmlPrincipal = path.join(path.resolve("../app/views/completes/"), "paginaPrincipal.ejs")
    res.render(htmlPrincipal)
  }
})

router.post("/login", async (req, res) => {
  const { email, password } = req.body

  if (!email || !password) {
    res.sendStatus(500)
  } else {
    const user = await comprobarLogin(email, password)
    if (user) {
      const response = await axios.get("http://localhost:3001/api/devolverTokens", {
        params: { email: email }
      })

      const tokens = response.data.tokens

      req.session.usuario = { email: email, tokens: tokens }
      res.redirect("/")
    } else {
      res.sendStatus(500)
    }
  }
})
```

```

export const comprobarLogin = async (email,password) => {
  try {
    const docRef = doc(db, "usuarios_f1", email.toLowerCase())
    const userDoc = await getDoc(docRef)

    if (!userDoc.exists()) {
      return null
    }

    const userData = userDoc.data()
    const hashedPassword = userData.password

    const isMatch = await bcrypt.compare(password, hashedPassword)

    if (!isMatch) {
      return null
    }

    return userData
  } catch (error) {
    console.error("Error en comprobarLogin:", error)
    return null
  }
}

```

Ahora vamos a explicar como funcionaria la aplicación en general a la hora de hacer cualquier tipo de consulta (GET,POST,PUT,DELETE)

Este seria el header que tenemos para nuestra aplicación en el que lo incluimos en todos nuestros distintos archivos .ejs y serviría para llevarnos a los distintos endpoints más importantes de la aplicación.

```

<nav>
  <a href="/insercion">Inserta información</a>
  <a href="/pilotos">Pilotos</a>
  <a href="/carreras">Carreras</a>
  <a href="/equipos">Equipos</a>
  <a href="/campeonatosPilotos">Campeonatos pilotos</a>
  <a href="/campeonatosConstructores">Campeonatos constructores</a>
  <a href="/perfil">Perfil</a>
  <a href="/logout">Cerrar sesión</a>
</nav>

```

Cuando vamos a insertar un piloto recogemos todos los datos que nos introduce en el formulario y la rutaFoto la ponemos nosotros en vacío para si en un futuro la quieren cambiar.

Cuando ya tenemos todo hacemos una llamada AXIOS a la API en la que introducimos todos los campos que necesita ese tipo de petición para que dentro de la API se ejecute la consulta que guarda el piloto en este caso pero seria lo mismo para los demás tipos de inserción.

```
router.post("/insertarPiloto", async (req,res) => {
  const { nombreCompleto, edad, nacionalidad, victorias, equipo, apodoAcronimo} = req.body
  const rutaFoto = ""
  try {
    await axios.put("http://localhost:3001/api/insertarPiloto", {
      nombreCompleto,
      edad,
      nacionalidad,
      victorias,
      equipo,
      apodoAcronimo,
      rutaFoto
    })
    res.redirect("insertarPiloto")
  } catch (error) {
    console.error("Error insertando piloto:", error)
    res.redirect("paginaPrincipal")
  }
})
```

```
router.put("/insertarPiloto", async (req, res) => {
  const { nombreCompleto, edad, nacionalidad, victorias, equipo, apodoAcronimo} = req.body

  const rutaFoto = ""

  try {
    await conn.execute(
      "INSERT INTO Pilotos (nombreCompleto, edad, nacionalidad, victorias, equipo, apodoAcronimo,rutaFoto) VALUES (?, ?, ?, ?, ?, ?, ?)",
      [nombreCompleto, edad, nacionalidad, victorias, equipo, apodoAcronimo,rutaFoto]
    )
    res.json({ message: "Piloto insertado correctamente" })
  } catch (error) {
    console.error("Error insertando piloto:", error)
    res.status(500).json({ message: "Error en la inserción" })
  }
})
```

Cuando pulsamos en el header a pilotos saldría la información de todos los pilotos disponibles dentro de la base de datos y se ejecutaría este código el cual haría una llamada AXIOS de tipo GET a la API y el resultado seria un JSON de todos los pilotos y lo pondríamos dentro de la plantilla pilotos.ejs.

```
router.get("/pilotos", async (req,res) => {
  if (comprobarSesion(req)) {
    try {
      const response = await axios.get("http://localhost:3001/api/pilotos")

      const pilotos = response.data

      res.render("pilotos.ejs", { pilotos })
    } catch (error) {
      res.render("index.ejs", { pilotos })
    }
  } else {
    res.render("index.ejs" ,{ pilotos })
  }
})
```



```
router.get("/pilotos", async (req, res) =>{
  let sql = "SELECT * FROM pilotos"
  const result = await conn.execute(sql)
  const todosPilotos = result.rows
  res.json(todosPilotos)
})
```

También dentro de esta propia página de pilotos podríamos hacer una búsqueda por un apodo de piloto, en la que cogemos el apodo buscado por el usuario y hacemos de nuevo una petición AXIOS a la API en la que se hace una consulta de tipo like para que sea mas sencillo para los usuarios la búsqueda y a parte de eso se haría la resta de los tokens por el tipo de consulta y si se falla en el tipo de apodo te llevaría a la misma pagina con todos los pilotos.

```
router.post("/buscarPiloto", async (req, res) => {
  const apodo = req.body.piloto
  const emailSession = req.session.usuario.email

  if (!comprobarSesion(req)) {
    return res.redirect("http://localhost:3000/pilotos")
  }
  try {
    const response = await axios.get("http://localhost:3001/api/buscarPiloto", {
      params: {apodo: apodo} //aquí paso el email de session porque dos awaits daban fallo entonces lo hacemos de la parte del servidor
    })
    //aquí no utilizamos el await porque no funcionaba y a parte porque entendemos
    //que hagas la consulta que hagas aunque sea errónea que te siga restando
    //porque consumes nuestra energia
    axios.put("http://localhost:3001/api/gestionarTokens", null, {
      params: { email: emailSession, numTokens: 5 }
    })

    const pilotos = response.data

    res.render("pilotos", { pilotos })
  } catch (error) { //si falla que se cargue todo otra vez
    console.error("Error en buscarPiloto:", error)
    const response = await axios.get("http://localhost:3001/api/pilotos")
    res.render("pilotos", { pilotos: response.data })
  }
})
```

```
router.get("/buscarPiloto", async (req, res) => {
  try {
    let { apodo } = req.query

    if (!apodo) {
      return res.status(400).json({ message: "Falta el apodo" })
    }

    const sql = "SELECT * FROM pilotos WHERE apodoAcronimo LIKE ?"
    const valores = [` ${apodo}%` ]

    const { rows: pilotos } = await conn.execute({ sql, args: valores })

    if (pilotos.length === 0) {
      return res.status(404).json({ message: "Piloto no encontrado" })
    }

    res.json(pilotos)
  } catch (error) {
    console.error("Error en buscarPiloto:", error)
    res.status(500).json({ message: "Error en el servidor" })
  }
})
```

Dentro de todos los divs o “cards” que utilizamos para mostrar los pilots hay siempre un botón para borrar o modificar dicho piloto.

```
<div class="accionesPiloto">
  <form action="/borrarPiloto" method="post">
    <input type="hidden" name="idPiloto" value="<%= piloto.id %>">
    <button type="submit">Borrar</button>
  </form>

  <form action="/editarPiloto" method="post">
    <input type="hidden" name="idPiloto" value="<%= piloto.id %>">
    <button type="submit">Modificar</button>
  </form>
</div>
```

Aquí cogemos el id del piloto a través del value y hacemos una llamada AXIOS a la API de tipo DELETE en la que pasamos el id de dicho piloto que hemos obtenido del body y borramos todo lo que tenga que ver con ese piloto si existe en la base de datos.

```
router.post("/borrarPiloto", async(req,res) => {
  const { idPiloto,passAdmin } = req.body
  if(comprobarSesion(req) && passAdmin === process.env.PASS_ADMIN){

    await axios.delete("http://localhost:3001/api/borrarPiloto",{
      data: { idPiloto }
    })

    res.redirect("/pilotos")
  } else {
    res.redirect("/index")
  }
})
```

```
router.delete("/borrarPiloto", async (req, res) => {
  const { idPiloto } = req.body

  if (!idPiloto) {
    return res.status(400).json({ message: "Falta el ID del piloto" })
  }

  try {
    await conn.execute("DELETE FROM pilotos WHERE id = ?", [idPiloto])
    res.json({ message: "Piloto eliminado correctamente" })
  } catch (error) {
    console.error("Error en la base de datos:", error)
    res.status(500).json({ message: "Error al eliminar el piloto" })
  }
})
```

Ahora para la modificación también lo obtendríamos el id desde el div o la card igual que arriba y haríamos un proceso parecido al de la eliminación.

Aquí lo que hacemos primero es recoger el id y ponerlo en una plantilla con los valores ya rellenos para así poder editarlo fácilmente.

```
router.post("/editarPiloto", async (req, res) => { //para mandarlo a la plantilla del piloto
  const { idPiloto, passAdmin } = req.body
  if (comprobarSesion(req) && passAdmin === process.env.PASS_ADMIN) {
    try {
      const respuesta = await axios.post("http://localhost:3001/api/obtenerPiloto", {
        idPiloto
      })

      const piloto = respuesta.data
      res.render("editarPiloto", { piloto })
    } catch (error) {
      console.error("Error al obtener el piloto:", error)
      res.redirect("/pilotos")
    }
  } else {
    res.redirect("/index")
  }
})
```

Y en esta parte es donde modificamos el piloto haciendo una llamada AXIOS de tipo PUT a la API pasandole todos los datos aunque algunos no sean nuevos para actualizar así toda la información necesaria de ese piloto.

```
router.post("/modificarPiloto", async (req, res) => { //modificar piloto como tal
  if (comprobarSesion(req)) {
    const { idPiloto, nombre, edad, nacionalidad, victorias, equipo, apodoAcronimo } = req.body

    try {
      await axios.put("http://localhost:3001/api/modificarPiloto", {
        idPiloto,
        nombre,
        edad,
        nacionalidad,
        victorias,
        equipo,
        apodoAcronimo
      })

      res.redirect("/pilotos")
    } catch (error) {
      console.error("Error al modificar el piloto:", error)
      res.redirect("/pilotos")
    }
  } else {
    res.redirect("/index")
  }
})
```

```
router.put("/modificarPiloto", async (req, res) => {

  const { idPiloto, nombre, edad, nacionalidad, victorias, equipo, apodoAcronimo } = req.body

  try {
    await conn.execute(
      "UPDATE pilotos SET nombreCompleto = ?, edad = ?, nacionalidad = ?, victorias = ?, equipo = ?, apodoAcronimo = ? WHERE id = ?",
      [nombre, edad, nacionalidad, victorias, equipo, apodoAcronimo, idPiloto]
    )
    res.json({ message: "Piloto modificado correctamente" })
  } catch (error) {
    console.error("Error en la base de datos:", error)
    res.status(500).json({ message: "Error al modificar el piloto" })
  }
})
```

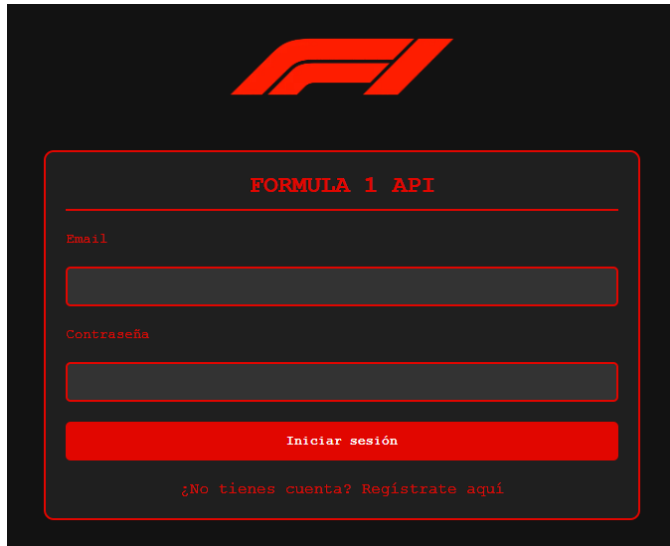
4. Ejecución de aplicación y explicación grafica

La forma en la que iniciamos todo nuestro proyecto seria de esta manera.

```
PS D:\proyectosNodeExpress\practica_ut7_ut8\app> node --env-file=../service/config.env .\app.mjs
Servidor corriendo en el puerto 3000

PS D:\proyectosNodeExpress\practica_ut7_ut8\service> node --env-file=../config.env .\api.mjs
Servidor corriendo en el puerto 3001
```

Lo primero que nos aparece al iniciar la aplicación es el login y una opción para si no tiene cuenta el usuario que le lleve a una página donde pueda registrarse.



The image shows a login form for 'FORMULA 1 API'. At the top is a red stylized 'F1' logo. Below it, the text 'FORMULA 1 API' is centered. There are two input fields: 'Email' and 'Contraseña' (Password). Below the password field is a red button labeled 'Iniciar sesión'. At the bottom, there is a link that says '¿No tienes cuenta? Regístrate aquí'.

Aquí nos lleva a la pagina principal donde tenemos en la parte de arriba las diferentes búsquedas y cosas que podemos hacer.

Y en la parte de debajo del header tenemos un video de presentación y mostramos cuantos tokens tenemos y también la parte del quiz en la que es una manera de ganar tokens para hacer diferentes consultas, si quisiéramos monetizar esto en vez de quiz seria una pagina con distintos precios para ganar tokens y se podría pagar a través de una pasarela de pago.



The image shows the main page of the Formula 1 API application. At the top is a navigation bar with links: 'Inserta información', 'Pilotos', 'Carreras', 'Equipos', 'Campeonatos pilotos', 'Campeonatos constructores', 'Perfil', and 'Cerrar sesión'. Below the navigation bar is a large video player showing a Formula 1 race. The video title is 'F1 1980s - The Era of Heroes'. Below the video player, it says 'Tus tokens son: 10015'. At the bottom, there is a quiz question: '¿Cuánto sabes sobre Fórmula 1? ¡Ponte a probar!' and a red button labeled 'Ir al Quiz'.

Cuando pulsamos en pilotos o cualquiera de los de arriba que nos muestre algún tipo de información primero nos muestra todos los disponibles y luego podemos a parte hacer una búsqueda que esta restaría tokens.

Lista de Pilotos

Piloto:

Fernando Alonso

Edad: 24

Nacionalidad: Español

Victorias: 7

Equipo: Mild Seven Renault F1 Team

Apodo/Acrónimo: ALO



Kimi Räikkönen

Edad: 26

Nacionalidad: Finlandés

Victorias: 9

Equipo: West McLaren Mercedes

Apodo/Acrónimo: RAI



Michael Schumacher

Edad: 36

Nacionalidad: Alemán

Victorias: 84

Equipo: Scuderia Ferrari Marlboro

Apodo/Acrónimo: MSC



Juan Pablo Montoya

Edad: 30

Nacionalidad: Colombiano

Victorias: 4

Equipo: West McLaren Mercedes

Apodo/Acrónimo: MOY



Giancarlo Fisichella

Edad: 32

Nacionalidad: Italiano

Victorias: 2

Equipo: Mild Seven Renault F1 Team

Apodo/Acrónimo: FIS



Ralf Schumacher

Edad: 30

Nacionalidad: Alemán

Victorias: 6

Equipo: Panasonic Toyota Racing

Apodo/Acrónimo: RSC



En la inserción de información primero nos pregunta qué tipo de inserción queremos hacer y la contraseña del admin y luego ya nos lleva a la pagina como tal para introducir todos los datos.

Piloto

Contraseña admin

Enviar

Volver

Nombre Completo:

Edad:

Nacionalidad:

Victorias:

0

Equipo:

Apodo o Acrónimo:

Insertar Piloto

Volver

También podemos visualizar nuestro perfil y podemos cambiar nuestra foto de perfil subiendo otra nueva, porque por defecto siempre hay una.

Perfil de Usuario



Cambiar Foto

Subir

Nombre Completo: prueba

Nombre de Usuario: prueba

Correo Electrónico: prueba@gmail.com

Tokens: 10010